Example Application: The Lunar Lander 🚀

To understand how Reinforcement Learning works in continuous state spaces, we will use the **Lunar Lander** simulation (a popular environment for RL researchers).

The Objective

You control a spaceship falling toward the moon.

- **Goal:** Fire thrusters to land safely between two yellow flags (the landing pad).
- **Failure:** Crashing into the surface.
- **Success:** A soft landing on the pad.

---

1. The Action Space (Discrete)

At every time step, the agent must choose exactly **one** of four possible actions:

1. **Do Nothing:** Let gravity and inertia pull the ship down.
2. **Fire Left Thruster:** Pushes the lander to the **Right**.
3. **Fire Main Engine:** Pushes the lander **Up** (against gravity).
4. **Fire Right Thruster:** Pushes the lander to the **Left**.

---

2. The State Space (Continuous)

The state s is a vector of **8 numbers** describing the lander's physical situation.

$$s = [x, y, \dot{x}, \dot{y}, \theta, \dot{\theta}, l, r]$$

- **Position:** x (Horizontal), y (Vertical).
- **Velocity:** $\dot{x}$ (Horizontal speed), $\dot{y}$ (Vertical speed).
- **Angle:** θ (Tilt), $\dot{\theta}$ (Angular velocity/spin).
- **Leg Sensors:**
  - l: Left leg touching ground? (Binary: 0 or 1).
  - r: Right leg touching ground? (Binary: 0 or 1).

---

3. The Reward Function

The designers of this environment carefully crafted the rewards to encourage smooth flying and fuel efficiency.

- **Good Rewards (+):**

    - Moving closer to the landing pad.
    - **+100** to **+140**: Successfully landing on the pad.
    - **+100**: Achieving a "soft landing" (not crashing).
    - **+10**: For each leg that touches the ground.
- **Bad Rewards (-):**
    - Moving away from the landing pad.
    - **-100**: Crashing.
    - **-0.3**: Firing the **Main Engine** (Fuel cost).
    - **-0.03**: Firing **Side Thrusters** (Fuel cost).

> ***Insight:*** *Designing the Reward Function is an art. It is much easier to specify* incentives *(e.g., "Don't waste fuel," "Don't crash") than to write a program specifying the exact thruster movements for every millisecond.*

Summary

- **Goal:** Learn a policy $\pi(s)$ that maps the 8 state variables to one of the 4 actions.
- **Metric:** Maximize the discounted return (using a high discount factor, e.g., $\gamma = 0.985$).